

Lab 4: Graphics

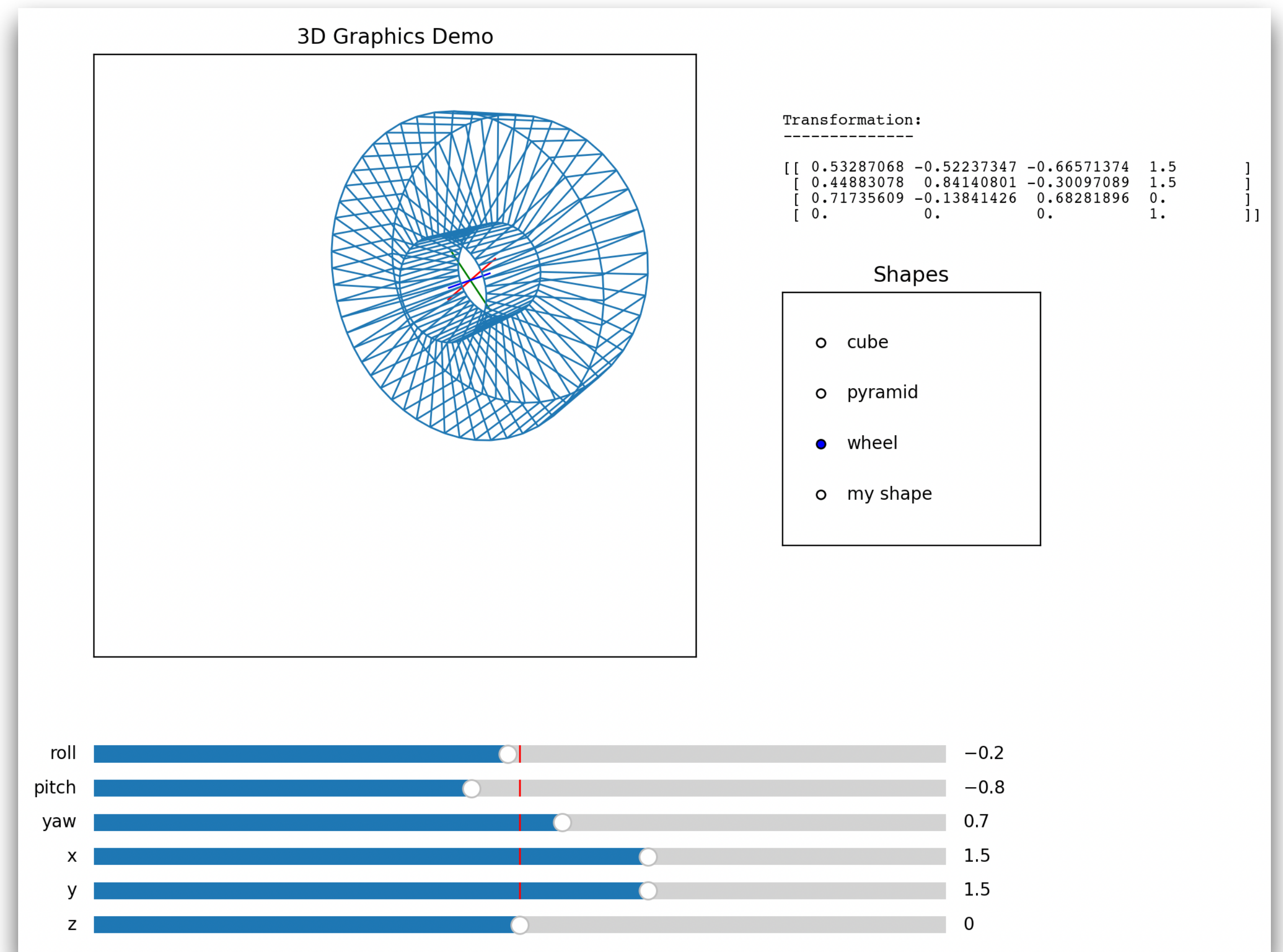
CAS CS 132: Geometric Algorithms

Teaching Fellow: Angelos Poulis (apoulis@bu.edu)

Mar. 23rd 2026

Goal: build an interactive 3D viewer

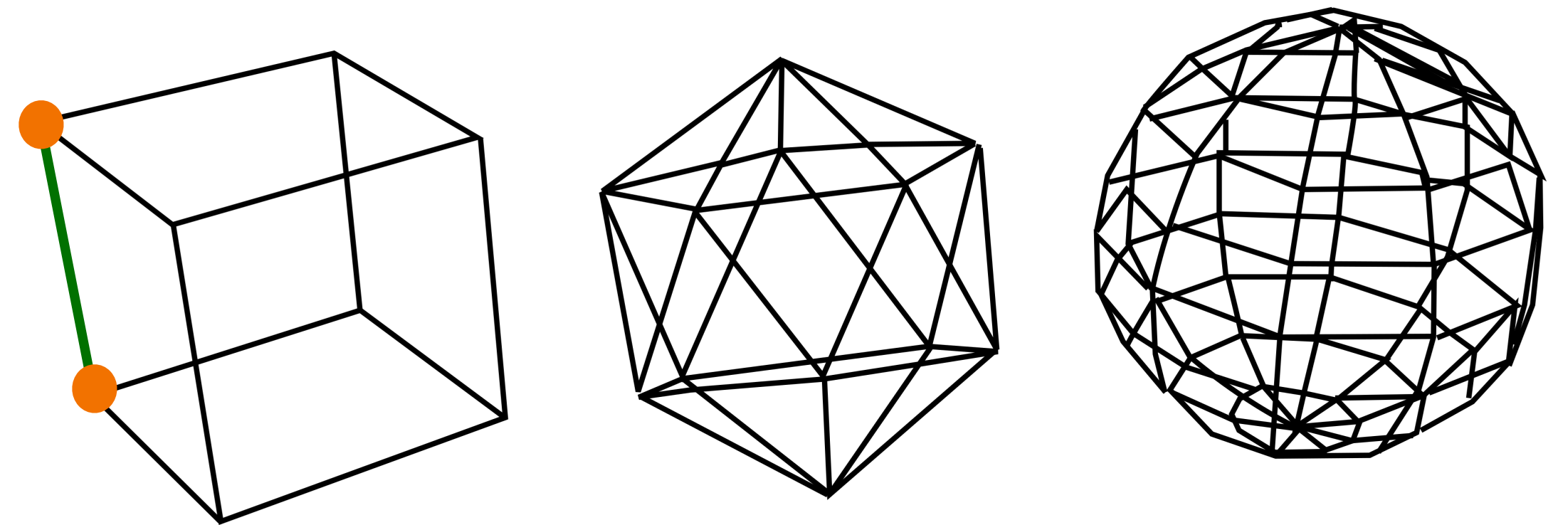
- **Pipeline:** 3D shape → pack into a matrix → rotate & translate → project to 2D → render
- **Every step is a matrix operation!**



How to represent a shape?

A wire-frame model

- A **point** in 3D space is defined by a triple of coordinates (x, y, z)
- A line **segment** is a pair of 3D points, i.e., $(p_1, p_2) = ((x_1, y_1, z_1), (x_2, y_2, z_2))$
- A **shape** is defined using many segments.



Read more: https://en.wikipedia.org/wiki/Wire-frame_model

Homogeneous coordinates

Because... translation is NOT a linear transformation in 3D :(

- You can't write $(x + t_x, y + t_y, z + t_z)$ as a 3×3 matrix times (x, y, z)

- **Fix:** add a 4-th coordinate, always 1

$t_{x/y/z}$: Offset of each coordinate

- Translation is now just matrix multiplication:

$$\begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ z + t_z \\ 1 \end{bmatrix}$$

Defining 3D rotation matrices

To Roll, Pitch and Yaw

Roll

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

leaves x alone; changes y, z

Pitch

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

leaves y alone; changes x, z

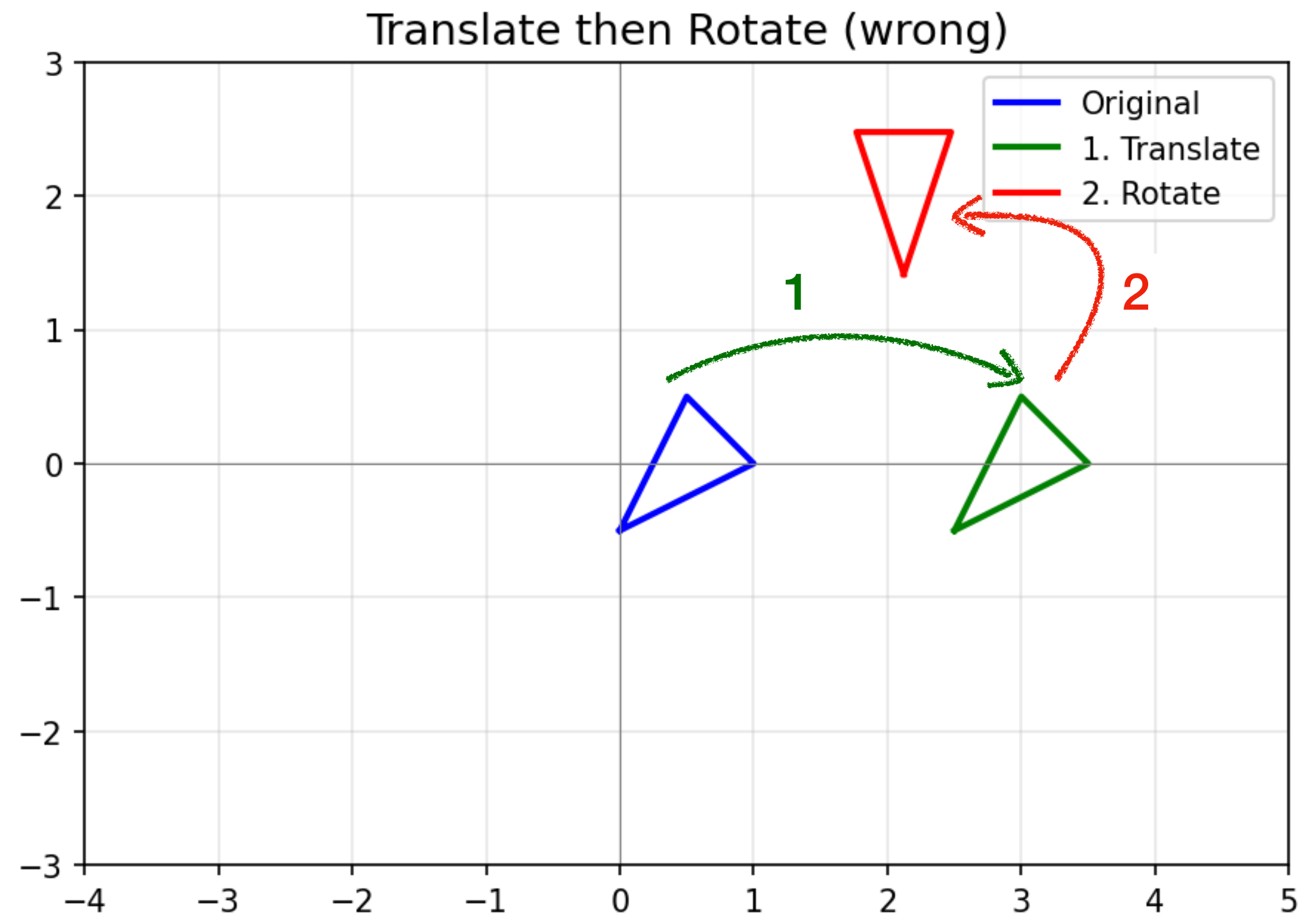
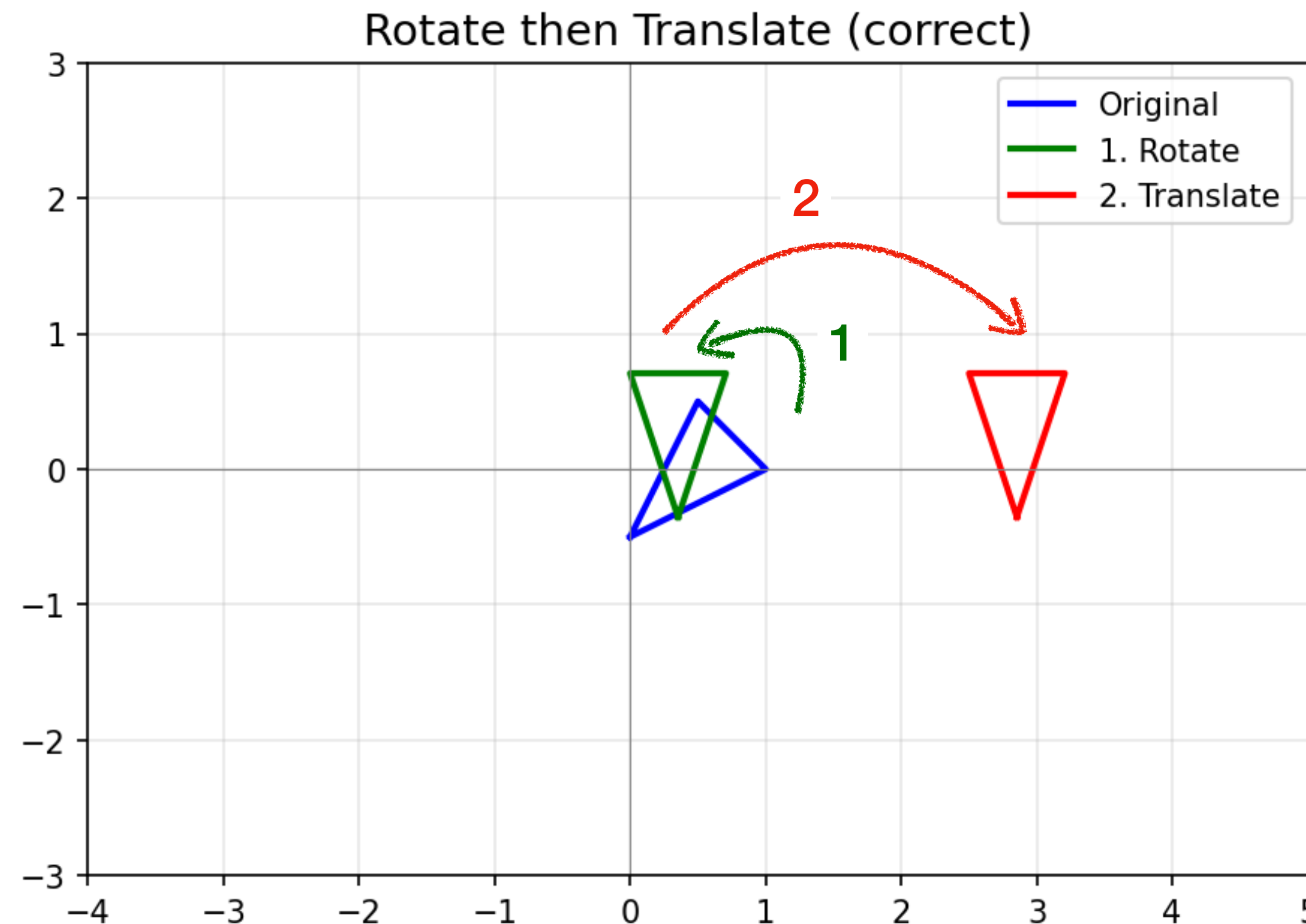
Yaw

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

leaves z alone; changes x, y

Order of operations

⚠ Matrix multiplication applies *right-to-left*: TR_v means **rotate** first, **translate** last



Left: spins in place, then moves. **Right:** moves along the origin, then rotates.

Perspective projection

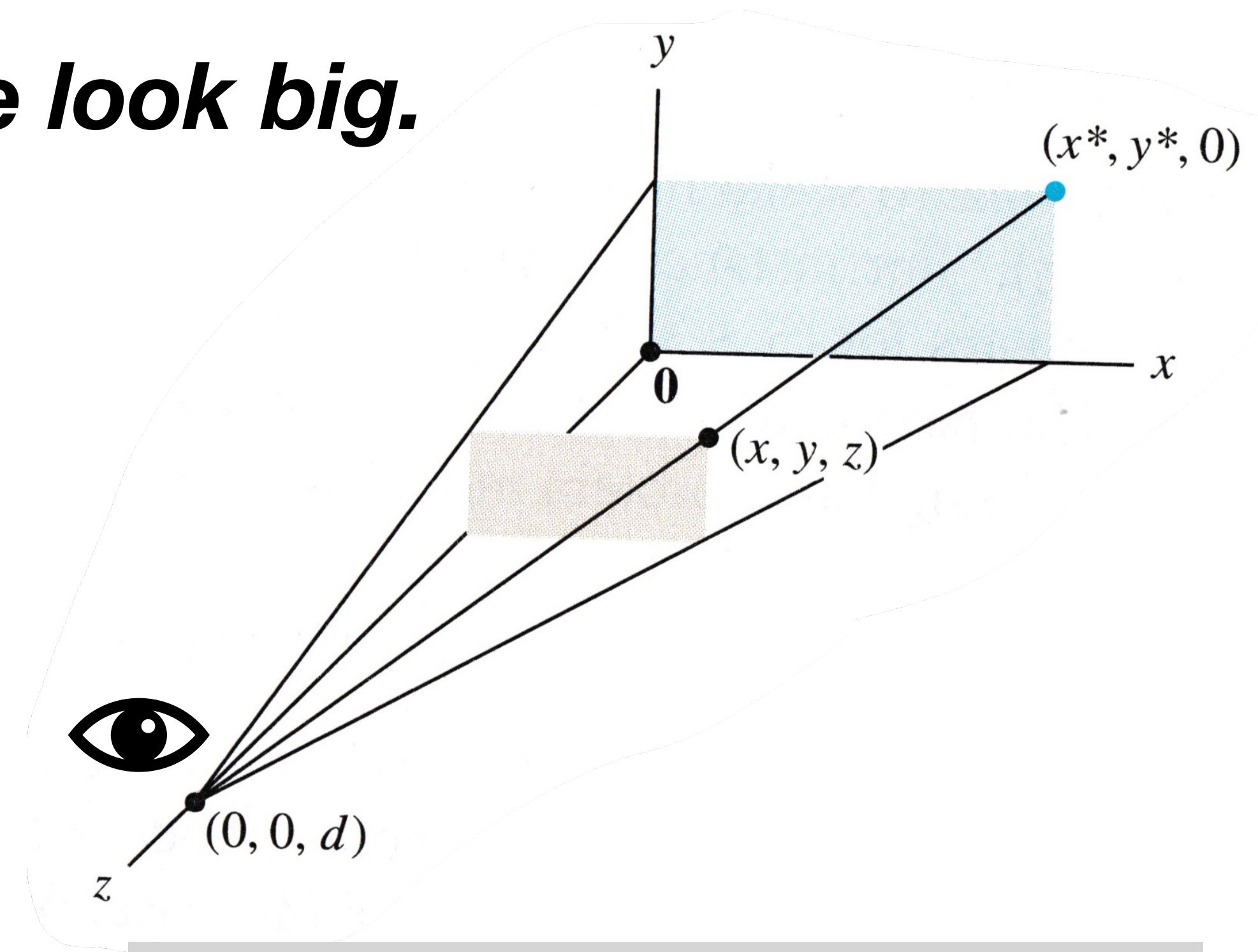
Things far away look small. Things close look big.

- A viewer (center of projection) seats at $(0,0,d)$ looking toward the origin
- Things close to the viewer should appear bigger, things far away smaller

$$x^* = \frac{x}{(1 - z/d)}, y^* = \frac{y}{(1 - z/d)}$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -1/d & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 0 \\ 1 - z/d \end{bmatrix} \xrightarrow{\text{divide by last}} \begin{bmatrix} x/(1 - z/d) \\ y/(1 - z/d) \end{bmatrix}$$

Drop last two coords; we only need x^*, y^*



🧐 Why do all this instead of just drop the z -coordinate?
✓ Because then depth would have no visual effect.

- **Link to Lab 4:** <https://nmmull.github.io/cs132-s26/labs/4/lab4.pdf>
- **Due:** April 2, 2026 by 8:00PM.